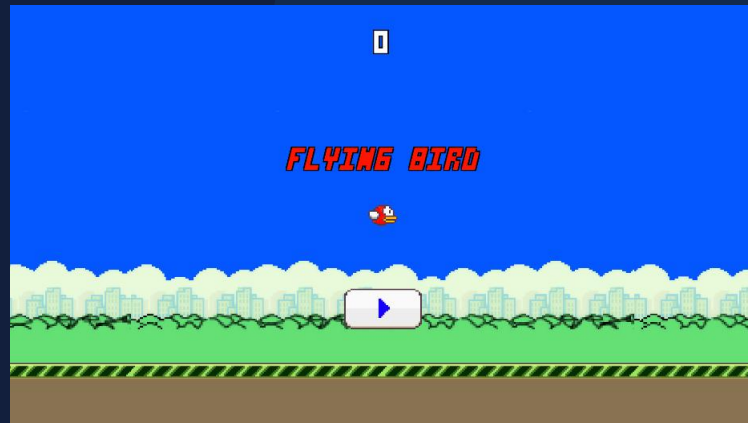


Flying Bird

プログラム説明資料



Unity

C#

2D Physics

Audio

横浜デジタルアーツ専門ゲーム科 3年 石川原 吉美

ジャンル:アクションゲーム

開発環境:C# / Unity

開発期間:2025年7月~8月

画面をタップしてジャンプし、パイプの間をくぐり抜けるカジュアルアクションゲーム。スマートフォン向けに設計し、シンプルな操作で直感的に楽しめるように制作しました。

FlyingBird

開発体制・使用ツールについて

01 所要時間の概算

- ・ 開発期間 約6～7週間（2025年7月～8月）
- ・ 週3～4回×2時間ペースで実装
- ・ 合計目安 約35～40時間

02 開発体制・担当範囲

- ・ 個人開発（1名）
- ・ 企画・設計・プログラム実装・デバッグ・サウンド組み込みまで全工程を担当

03 学校のFW・AI活用範囲

- ・ 学校指定の共通フレームワークは不使用、ゼロから構築
- ・ 実装で不明な点やエラー発生時にAIに質問し参考にした
- ・ ゲームロジックの設計・実装は自身で実施

04 外部アセット・ミドルウェア

- ・ 鳥・パイプ等のグラフィックとBGM/SEはフリー素材を使用し、Photoshopで加工
- ・ ミドルウェア・外部SDKの組み込みはなし

アピールポイント

01 GameManager 設計

シングルトンパターン
による統一的なゲーム
進行管理

02 Player 物理挙動

重力・ジャンプ・スプ
ライトアニメーション
の実装

03 Pipes 生成と削除

画面外で自動破棄され
るメモリ効率の高いオ
ブジェクト管理

04 Spawner スポーン制御

ランダム高度でパイプ
を一定間隔で連続生成

05 SoundManager音響設計

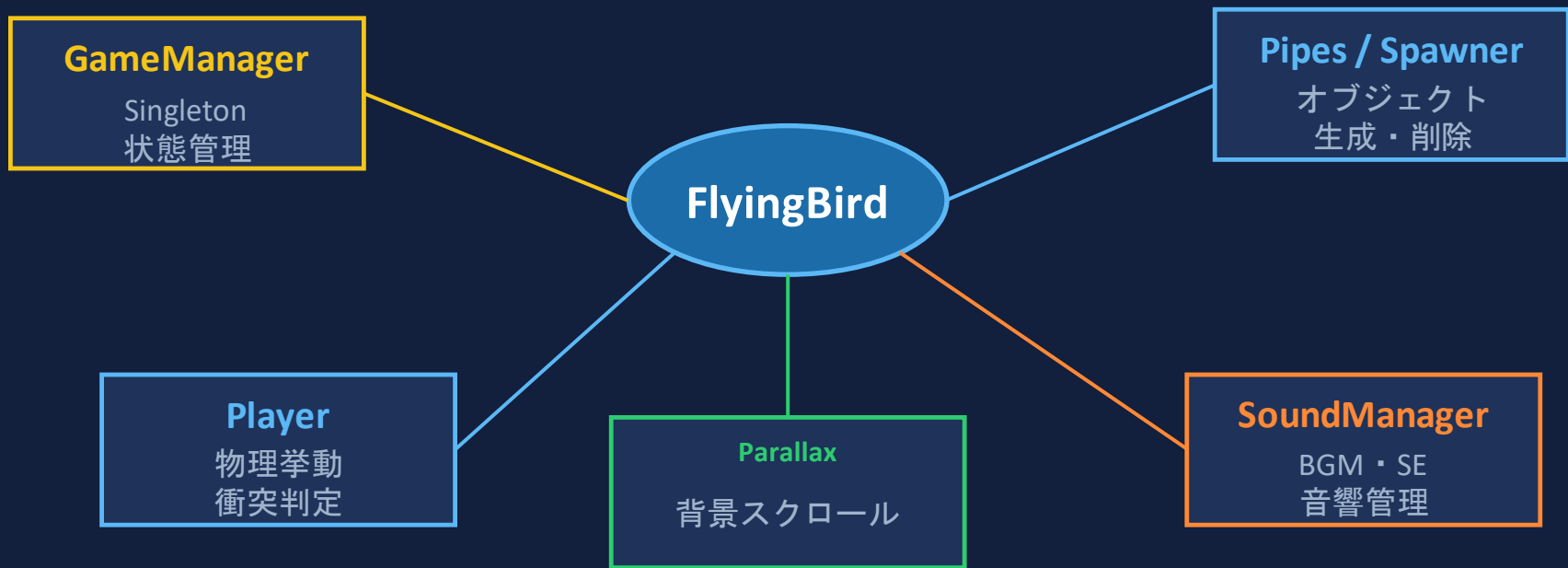
BGM・SE・ゲームオー
バー音を独立管理する
音響システム

06 Parallax 背景演出

テクスチャオフセット
で実現するスムーズな
視差スクロール

ゲームシステムー アーキテクチャ全体図

アーキテクチャ



各コンポーネントはGameManager を介してゲーム状態を共有。
SoundManager はSingleton で全体から呼び出し可能。

GameManager 設計

ゲーム進行管理

シングルトン実装

Instance プロパティで他スクリプトからどこからでもアクセス可能

UI制御の集約

scoreText ・ titleText ・ playButton ・ gameOver を一元管理

ゲーム状態遷移

Title → Play → GameOver の状態をメソッドで明示的に切り替え

Time.timeScale制御

Pause/Resume をtimeScaleで実現し再利用性の高い停止処理

// GameManager.cs 抜粋

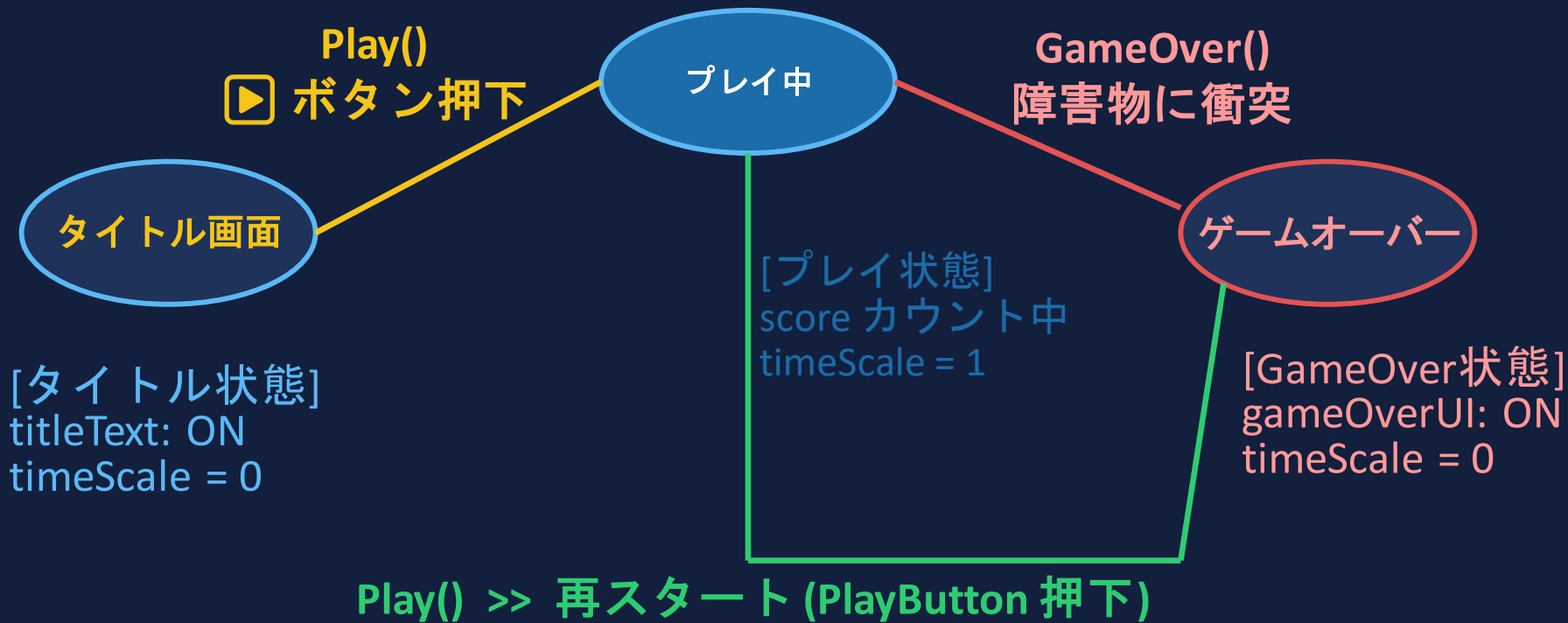
```
public static GameManager Instance
{ get; private set; }

private void Awake()
{
    if (Instance == null)
        Instance = this;
    else {
        Destroy(gameObject);
        return;
    }
    Application.targetFrameRate = 60;
    Title(); // タイトル画面初期化
}

public void Play()
{
    SoundManager.Instance
        .StopGameOverSE();
    score = 0;
    Time.timeScale = 1f;
    player.StartGame();
}
```

GameManager — 状態遷移図

ステートマシーン



Pause(): Time.timeScale=0, player.enabled=false | Resume(): Time.timeScale=1

Player 物理挙動

操作・アニメーション

ジャンプ力

strength = 5f
タップ入力で
Vector3.up * strength を
direction に設定

重力加速度

gravity = -9.8f
毎フレーム加算し自然
な落下を実現

傾き表現

tilt = 5f * direction.y
z軸回転で速度に応じ
た見た目を演出

スプライトアニメ

0.15秒ごとにコマ切り
替え、InvokeRepeating
でループ再生

衝突処理: OnTriggerEnter2D

Obstacle タグ

BGM停止・
SE再生

GameOver()

Scoring タグ → IncreaseScore() でスコア加算

// Player.cs 抜粋

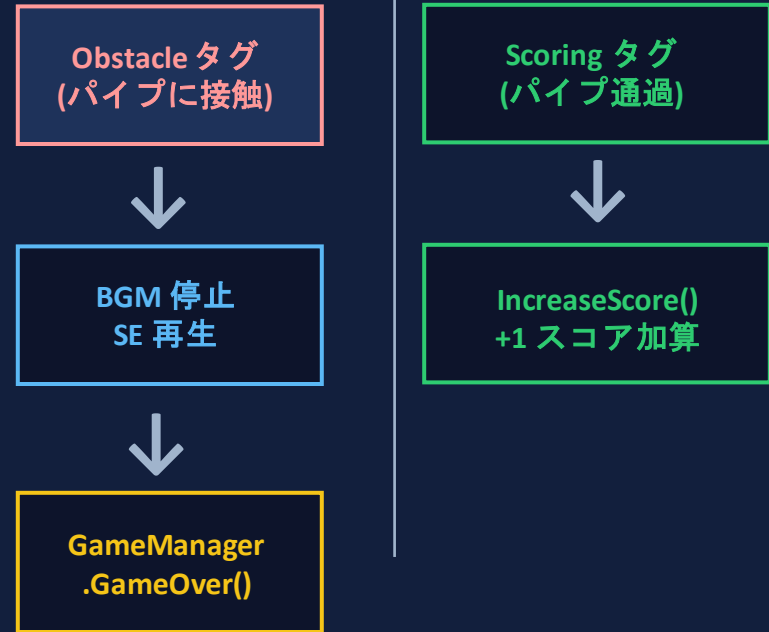
```
private void Update()
{
    // タップ or スペースキー
    if (Input.GetMouseButtonDown(0))
    {
        direction = Vector3.up * strength;
        SoundManager.Instance
            .PlaySE(jumpClip);
    }
    // 重力加算
    direction.y += gravity *
Time.deltaTime;
    transform.position +=
        direction * Time.deltaTime;
    // 傾き反映
    Vector3 rot = transform.eulerAngles;
    rot.z = direction.y * tilt;
    transform.eulerAngles = rot;
}
```

Player — 物理挙動 図解

物理挙動



衝突処理: OnTriggerEnter2D



スプライトアニメ: `InvokeRepeating` で 0.15 秒ごとにコマ切り替え

Pipes & Spawner — オブジェクト管理

生成・移動・削除

Pipes.cs — パイプ移動と自動削除

- speed = 5f で毎フレーム左移動
- Camera.main.ScreenToWorldPoint で画面左端座標を動的に取得
- 画面外に出た瞬間 Destroy(gameObject) でメモリを即時解放

設計ポイント:

InvokeRepeating

Destroy

Random.Range

Spawner.cs — パイプ連続生成

```
// Spawner.cs
// OnEnable でリピート開始
InvokeRepeating(
    nameof(Spawn),
    spawnRate, spawnRate);

private void Spawn()
{
    GameObject p = Instantiate(
        prefab,
        transform.position,
        Quaternion.identity);
    // ランダムY位置を適用
    p.transform.position +=
        Vector3.up *
        Random.Range(
            minHeight, // -1f
            maxHeight); // 3.5f
}
```

Pipes / Spawner — ライフサイクル図

土管ライフサイクル



← パイプは毎フレーム左へ移動。画面外(LeftEdge)を超えた瞬間Destroy()でメモリ解放

SoundManager — 音響設計

BGM・SE管理

音響アーキテクチャ

bgmSource

BGM専用 AudioSource
ループ再生・停止を管理

seSource

SE共用 AudioSource
PlayOneShot で重ね再生

GameOver SE

なぜ GameOverSE だけ Stop 可能にしたか?

再スタート時に前回のゲームオーバー音が残らないように seSource.clip == gameOverClip のチェックで安全に Stop している。PlayOneShot は停止できないため意図的に通常の Play() を使用。

// SoundManager.cs 抜粋

```
// 通常SE: 重ね再生可能 停止不可
public void PlaySE(AudioClip clip)
{
    seSource.PlayOneShot(clip);
}

// GameOver音: 停止可能
public void PlayGameOverSE()
{
    seSource.clip = gameOverClip;
    seSource.loop = false;
    seSource.Play();
}

// 再スタート時に呼出
public void StopGameOverSE()
{
    if (seSource.isPlaying &&
        seSource.clip == gameOverClip)
    {
        seSource.Stop();
    }
}
```

Parallax — 背景スクロール演出

視差スクロール

仕組み

1

MeshRenderer 取得

Awake() GetComponent<MeshRenderer>() をキャッシュ

2

テクスチャオフセット

material.mainTextureOffset の X 値を
毎フレーム加算

3

速度調整

animationSpeed × Time.deltaTime で
フレームレート非依存のスクロール

メリット: 追加オブジェクトなしで無限ループ
する背景を実現。軽量で GPU 負荷も低い。

// Parallax.cs

```
public class Parallax : MonoBehaviour
{
    private MeshRenderer meshRenderer;
    public float animationSpeed = 1f;

    private void Awake()
    {
        // MeshRenderer をキャッシュ
        meshRenderer =
            GetComponent<MeshRenderer>();
    }

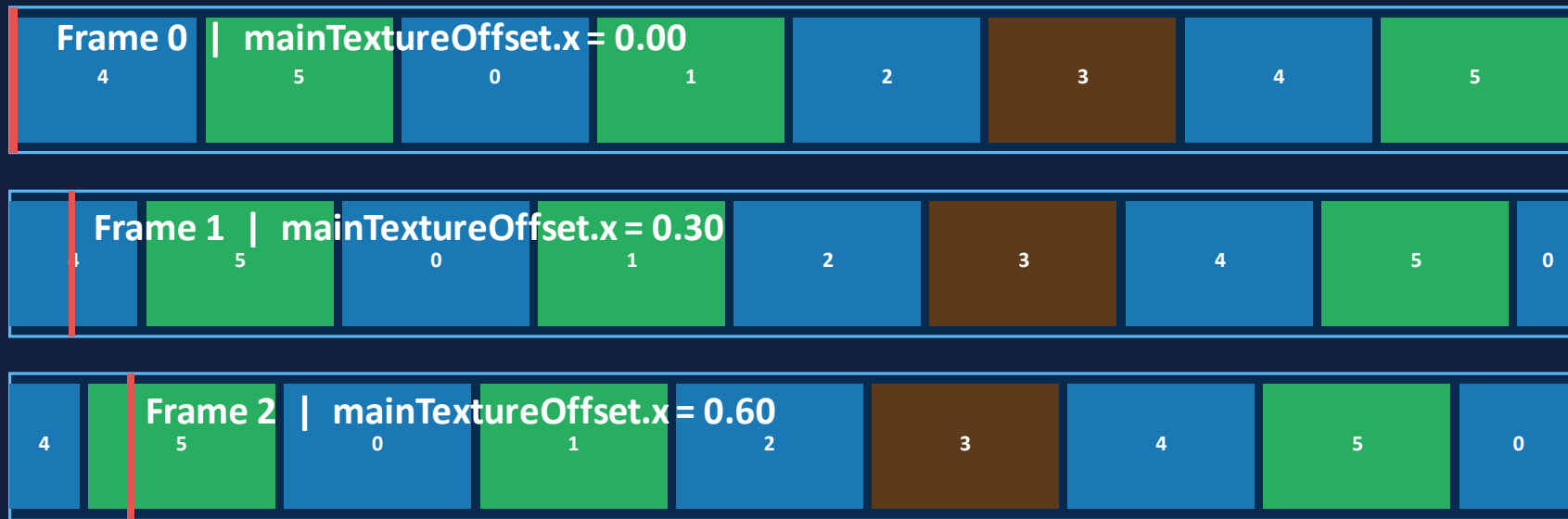
    private void Update()
    {
        // 現在オフセットを取得
        Vector2 offset =
            meshRenderer.material
                .mainTextureOffset;

        // X方向にスクロール加算
        offset += new Vector2(
            animationSpeed * Time.deltaTime,
            0);

        // マテリアルに反映
        meshRenderer.material
            .mainTextureOffset = offset;
    }
}
```

Parallax — テクスチャスクロール概念図

スクロール概念



◀ ◀ 見かけ上のスクロール方向（左へ移動しているように見える）

仕組み: mainTextureOffset の X 値を毎フレーム加算 → UVが右にシフト → テクスチャが左へ流れて見える

利点: 追加オブジェクト不要・無限ループ・GPU負荷が低い（頂点移動なし）

まとめ

GameManager

シングルトンによる状態管理で、UIとゲーム進行を一元コントロール

Player

カスタム重力実装により、UnityのPhysics非依存で軽量かつ調整しやすい物理挙動を実現

Pipes/Spawner

画面外の即時Destroyとランダム生成で、メモリ効率とゲームバランスを両立

SoundManager

PlayOneShot と Play/Stopを使い分け、ゲームオーバー音のみ停止可能な設計

Parallax

テクスチャオフセットのみで無限スクロールを実現し、描画コストを最小化